Implementando la Neurona M-P

October 12, 2026

Implementando la Neurona M-P

April 7, 2026

Caso Práctico: Neurona de McCulloch y Pitts

Kevin Xavier Suasnavas Villarreal

Link a mi GitHub

https://github.com/kevinsuas433/Machine2026.git

---

1 1. Caso Práctico: Neurona de McCulloch y Pitts

1.1 1.1. Implementación de la MPNeuron

En esta práctica vamos a implementar una neurona artificial de McCulloch y Pitts utilizando Python y la librería NumPy.

La neurona recibe entradas binarias (0 y 1), suma sus valores y compara el resultado con un umbral (*threshold*). Si la suma es mayor o igual al umbral, la neurona devuelve `True`; en caso contrario, devuelve `False`.

El objetivo es comprender cómo funciona una neurona artificial sencilla mediante la programación de una clase, sus métodos y algunos ejemplos de predicción.

[1]:
```python
# Importamos NumPy para trabajar con arreglos y almacenar las predicciones.
import numpy as np


# Creamos la clase MPNeuron que representa una neurona artificial.
class MPNeuron:

    # Inicializamos la neurona sin asignarle todavía un umbral.
    def __init__(self):
        self.threshold = None

    # Definimos el funcionamiento de la neurona para una entrada.
    def model(self, x):
        # x contiene las entradas binarias, por ejemplo [1, 0, 1, 0].
```

```
        # Sumamos todas las entradas para obtener el valor de activación.
        z = sum(x)

        # Activamos la neurona si la suma es mayor o igual al umbral.
        return (z >= self.threshold)

    # Creamos un método para realizar predicciones con varias entradas.
    def predict(self, X):
        # X contiene una lista de diferentes combinaciones de entradas.
        Y = []

        # Recorremos cada combinación de entradas.
        for x in X:
            # Evaluamos la entrada utilizando el modelo de la neurona.
            result = self.model(x)

            # Guardamos el resultado de cada predicción.
            Y.append(result)

        # Convertimos los resultados en un arreglo de NumPy y lo devolvemos.
        return np.array(Y)
```

La clase `MPNeuron` representa una neurona artificial sencilla.

- `__init__()`: inicializa el atributo `threshold`, que representa el umbral de activación.
- `model(x)`: suma las entradas y determina si la neurona se activa.
- `predict(X)`: procesa varias combinaciones de entradas y devuelve sus resultados.

En este modelo, las entradas son binarias. La salida será `True` cuando la suma alcance el umbral establecido y `False` cuando no lo alcance.

[5]:
```
# Creamos un objeto de la clase MPNeuron.
# Este objeto nos permitirá utilizar los métodos definidos anteriormente.
mp_neuron = MPNeuron()
```

En esta sección creamos una instancia de la clase `MPNeuron`. Esto significa que construimos un objeto que representa nuestra neurona artificial y que podremos configurar para realizar predicciones.

[6]:
```
# Establecemos el umbral de activación de la neurona en 4.
# La neurona se activará cuando la suma de las entradas sea mayor o igual a 4.
mp_neuron.threshold = 4
```

El umbral determina cuántas entradas activas necesita la neurona para producir una salida verdadera.

En este ejercicio, el umbral es `4`. Como cada entrada puede valer 0 o 1 y tenemos cuatro entradas, la neurona solamente devolverá `True` cuando todas sean iguales a 1.

[7]:
```
# Evaluamos tres combinaciones diferentes de entradas binarias.
# La primera tiene una entrada activa, la segunda tiene cuatro
```

```python
# y la tercera tiene tres.
predicciones = mp_neuron.predict([
    [1, 0, 0, 0],
    [1, 1, 1, 1],
    [1, 1, 1, 0]
])

# Mostramos las predicciones obtenidas por la neurona.
print(predicciones)
```

```
[False  True False]
```

1.2 Interpretación de los resultados

Los resultados permiten comprobar el funcionamiento de la neurona de McCulloch y Pitts:

- **Primera entrada:** `[1, 0, 0, 0]`. La suma es 1, por lo que no alcanza el umbral de 4 y devuelve `False`.
- **Segunda entrada:** `[1, 1, 1, 1]`. La suma es 4, alcanza el umbral y devuelve `True`.
- **Tercera entrada:** `[1, 1, 1, 0]`. La suma es 3, no alcanza el umbral y devuelve `False`.

Por lo tanto, la neurona funciona correctamente de acuerdo con la condición definida en el método `model()`.

[ ]: